

Ingeniería de Software I

Laboratorio # 1 – Flujo colaborativo con GitHub y Docker para implementar una API REST

Fecha de entrega: miércoles 16 de septiembre de 2026.

Docente: Juan Pablo Bustamante Moreno.

Escuela Tecnológica Instituto Técnico Central (ETITC).

En este laboratorio construirán una API REST sencilla con operaciones GET, POST, PUT y DELETE, conectada a una base de datos ejecutada mediante Docker. El énfasis no está en desarrollar una aplicación compleja, sino en aplicar un flujo técnico colaborativo: GitHub, ramas, Pull Requests, revisión entre pares, resolución de conflictos y ejecución reproducible con Docker.

La aplicación tendrá poca complejidad funcional. La idea es que puedan configurar, versionar, integrar y ejecutar una solución compuesta por una API y una base de datos, utilizando contenedores. Esto fortalece la aplicación de herramientas y prácticas asociadas a la planeación, desarrollo y mantenimiento de software.

Objetivo

Este laboratorio se desarrollará **en parejas**. Al finalizar, cada estudiante habrá adquirido las herramientas para:

- Trabajar colaborativamente sobre un repositorio GitHub.
- Crear ramas de funcionalidad y Pull Requests.
- Realizar revisiones cruzadas de cambios.
- Generar y resolver un conflicto obligatorio de Git.
- Construir una API REST mínima con operaciones GET, POST, PUT y DELETE.

- Conectar la aplicación a una base de datos.
- Ejecutar la API y la base de datos mediante Docker Compose.
- Utilizar variables de entorno para configurar la conexión.
- Mantener la información de la base de datos usando un volumen Docker.
- Registrar el proceso técnico, decisiones y evidencias en un documento PDF.

Caso del laboratorio

Construyan una API REST sencilla para administrar **notas de trabajo de un equipo**.

La API puede llamarse:

```
team-notes-api
```

Cada nota representa una anotación corta registrada por un integrante del equipo.

Pueden cambiar el contexto de la API por otro dominio sencillo —tareas, libros, mascotas, hábitos, películas, productos o reservas—, siempre que conserven el mismo nivel de complejidad: una entidad principal, operaciones CRUD básicas, validaciones mínimas y persistencia en base de datos.

1. Requisitos funcionales de la API

La API debe administrar notas con los siguientes campos mínimos:

Campo	Tipo sugerido	Descripción
id	Entero/autogenerado	Identificador único
title	Texto corto	Título de la nota
content	Texto	Contenido de la nota
author	Texto corto	Autor de la nota
created_at	Fecha y hora	Fecha de creación

La API debe incluir los siguientes endpoints:

Operación	Método	Ruta	Descripción
Estado del servicio	GET	/health	Informa que la API está activa
Listar notas	GET	/notes	Devuelve todas las notas
Consultar nota	GET	/notes/{id}	Devuelve una nota según identificador
Crear nota	POST	/notes	Registra una nueva nota
Actualizar nota	PUT	/notes/{id}	Actualiza una nota existente
Eliminar nota	DELETE	/notes/{id}	Elimina una nota existente

La API debe responder en formato JSON.

Validaciones mínimas

- `title`, `content` y `author` son campos obligatorios; no pueden estar vacíos.
- El identificador debe ser generado por la base de datos o por la aplicación de manera consistente.
- `created_at` debe registrarse al crear la nota.
- Al solicitar una nota inexistente, se debe responder con código 404.
- Al crear o actualizar con datos inválidos, se debe responder con código 400.
- Las respuestas de error deben ser claras y no pueden exponer datos internos.

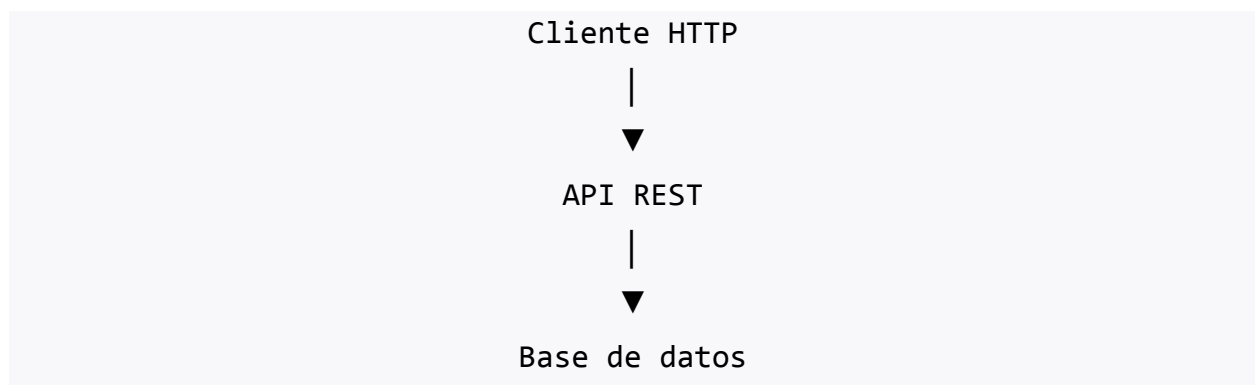
Códigos HTTP esperados

Situación	Código esperado
Consulta exitosa	200 OK

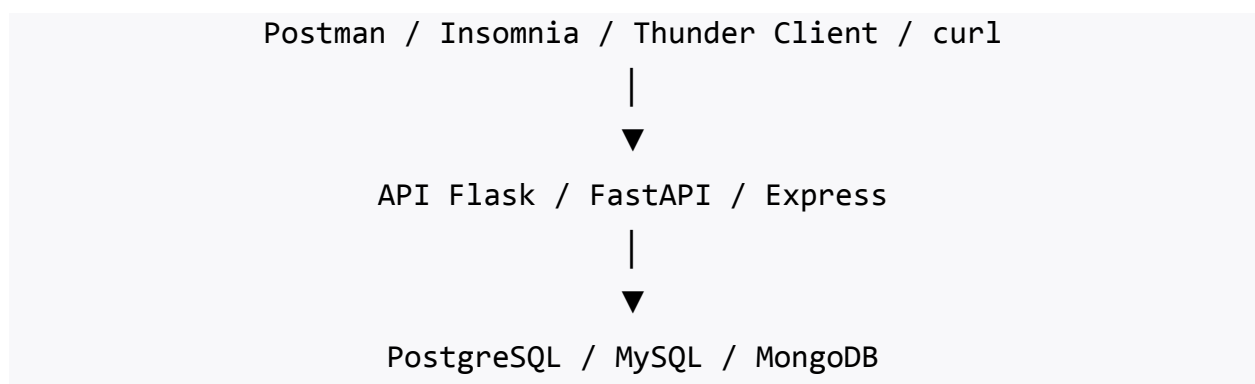
Creación exitosa	201 Created
Actualización exitosa	200 OK
Eliminación exitosa	200 OK, o bien 204 No Content
Datos inválidos	400 Bad Request
Nota inexistente	404 Not Found
Error interno	500 Internal Server Error

2. Arquitectura mínima requerida

La solución debe contener dos servicios ejecutándose con Docker Compose:



Ejemplo:



La API debe conectarse a la base de datos usando variables de entorno.

3. Desarrollo paso a paso

Paso 1. Definir la tecnología

La pareja puede elegir libremente las tecnologías, pero se recomienda que usen las que utilizarán en su proyecto, para que puedan irse familiarizando (ej. Python con Flask/FastAPI + PostgreSQL, Node.js con Express + MongoDB, etc). La opción seleccionada debe quedar indicada en el PDF de seguimiento.

Paso 2. Crear el repositorio en GitHub

Responsable inicial: Estudiante A

1. Cree un repositorio público con el nombre:

```
lab1-apellidoA-apellidoB
```

2. Agregue al Estudiante B como colaborador.
3. Cree las ramas:

```
main
```

```
develop
```

4. Proteja la rama `main`, si la configuración del repositorio lo permite:
 - Debe requerir Pull Request antes de fusionar.
 - No debe permitir pushes directos.
5. Cree un README sencillo con:

```
# Laboratorio 1 – GitHub, Docker y API REST
```

```
## Integrantes
```

- Nombre completo – usuario GitHub
- Nombre completo – usuario GitHub

```
## Descripción
```

```
API REST sencilla para gestionar notas.
```

Tecnologías

Lista de tecnologías usadas.

6. Agregue un `.gitignore` apropiado para la tecnología seleccionada.
7. Agregue un `.dockerignore`.
8. Cree un archivo `.env.example` con las variables necesarias para la API y la base de datos.
9. Verifique que `.env` esté incluido en `.gitignore`.
10. Suba la estructura inicial a `main`.
11. Cree `develop` a partir de `main`.

Paso 3. Crear la configuración de entorno

Incluyan un archivo:

```
.env.example
```

Ejemplo para PostgreSQL:

```
APP_ENV=development
APP_PORT=8000

DB_HOST=db
DB_PORT=5432
DB_NAME=team_notes
DB_USER=team_notes_user
DB_PASSWORD=change_this_password
```

Cada integrante debe:

1. Crear un archivo `.env` local a partir de `.env.example`.
2. Usar las variables de entorno para configurar la conexión de la API a la base de datos, cambiando al menos el valor de una variable para su ejecución local (por ejemplo, `DB_PASSWORD`).

Paso 4. Crear Dockerfile y Docker Compose

La pareja debe crear un `Dockerfile` para la API, de acuerdo con la tecnología que eligieron. Este archivo debe:

- Usar una imagen oficial del lenguaje.
- Definir un directorio de trabajo.
- Copiar archivos de dependencias antes del código, si aplica.
- Instalar dependencias.
- Copiar el código de la API.
- Exponer el puerto de la aplicación y ejecutar la API.

Luego, deben crear un archivo:

```
compose.yaml
```

Este archivo debe definir, como mínimo:

- Servicio `api`.
- Servicio `db`.
- Uso de archivo `.env`.
- Mapeo de puerto para la API.
- Variables de entorno para la base de datos.
- Volumen Docker para persistencia de datos.
- Dependencia de la API respecto al servicio de base de datos.

Estructura de ejemplo:

```
services:
  api:
    build: .
    env_file: .env
    ports:
      - puerto_host:puerto_contenedor
    depends_on:
      - db
```

```
db:
  image: imagen_base_de_la_bd
  environment:
  variables_de_base_de_datos
  volumes:
    - volumen_de_datos:ruta_de_datos_de_la_bd
```

```
volumes:
  volumen_de_datos:
```

La sintaxis concreta dependerá de la base de datos seleccionada.

Paso 5. Crear la base de datos

La base de datos debe contener una tabla o colección para las notas.

Si usan SQL, la tabla debe tener una estructura equivalente a:

Campo	Tipo sugerido	Condición
id	Entero	Llave primaria autogenerada
title	Texto corto	Obligatorio
content	Texto	Obligatorio
author	Texto corto	Obligatorio
created_at	Fecha/hora	Obligatorio

Pueden crear la tabla mediante script SQL de inicialización, migración, ORM, código de inicialización de la aplicación... la estrategia elegida debe quedar explicada en el PDF.

Si usan MongoDB, la colección debe almacenar los campos definidos y aplicar validación desde la API.

Paso 6. Implementar la API por ramas

No se permite implementar directamente en `main` o `develop`.

Contribución del Estudiante A

El Estudiante A debe crear una rama desde `develop`, por ejemplo:

`feature/read-notes`

Debe implementar:

- `GET /health`

Es un endpoint que debe responder si la API funciona. Por ejemplo:

```
{
  "status": "ok",
  "environment": "development"
}
```

- `GET /notes`

Retorna una lista de objetos JSON con todas las notas que existen, o una lista vacía si no hay notas.

- `GET /notes/{id}`

Retorna la nota solicitada. Si no existe, debe responder con un mensaje de error.

Después, el estudiante debe:

1. Hacer al menos dos commits descriptivos (con mensajes en inglés).
2. Subir la rama a GitHub.
3. Crear Pull Request hacia `develop`.
4. Solicitar revisión al Estudiante B, que hará mínimo un comentario útil de revisión.
5. Corregir o justificar la observación.
6. Fusionar el Pull Request hacia `develop`.

Contribución del Estudiante B

El Estudiante B debe crear una rama desde `develop`, por ejemplo:

```
feature/write-notes
```

Debe implementar:

- `POST /notes`

La aplicación debe validar que los campos obligatorios existan, generar automáticamente un `id` y un `created_at`, y por último debe guardar la nota en base de datos.

- `PUT /notes/{id}`

Sigue el mismo flujo de la anterior, pero ahora se debe actualizar una nota existente. Si no existe la nota, retorna un mensaje de error (o la inserta, pueden elegir lo que consideren más conveniente).

- `DELETE /notes/{id}`

Sigue el mismo flujo, pero ahora se debe eliminar una nota existente. Si no existe la nota, retorna un mensaje de error

Después, el estudiante debe:

1. Hacer al menos dos commits descriptivos (con mensajes en inglés).
2. Subir la rama a GitHub.
3. Crear Pull Request hacia `develop`.
4. Solicitar revisión al Estudiante A, que hará mínimo un comentario útil de revisión.
5. Corregir o justificar la observación.
6. Fusionar el Pull Request hacia `develop`.

Para ir registrando sus cambios en commits, pueden utilizar la siguiente secuencia de comandos:

```
git add .  
git commit -m "feat: add endpoint /health"
```

Para subir la rama a github, pueden usar:

```
git push -u origin feature/health-and-read-notes
```

Paso 7. Crear un conflicto en el merge

Si durante el desarrollo de la API en los pasos anteriores ya les surge un conflicto de manera orgánica al fusionar los cambios, pueden omitir este paso y continuar con la resolución de dicho conflicto. De lo contrario, procedan con:

1. Estudiante A crea una rama:

```
feature/conflict-student-a
```

2. Estudiante B crea una rama:

```
feature/conflict-student-b
```

3. Ambos modifican la misma sección del código (puede ser del README.md), pero cada integrante realiza un cambio diferente.
4. Cada integrante hace commit y push de su rama.
5. Crean Pull Requests hacia develop.
6. Fusionan primero el Pull Request del Estudiante A.
7. Intentan fusionar el Pull Request del Estudiante B.
8. GitHub debe detectar el conflicto.

Paso 8. Resolver el conflicto

Responsable principal: Estudiante B

1. Actualice su rama con los cambios de develop.
2. Identifique las marcas de conflicto.

3. Edite el archivo para conservar una versión coherente (debe tomar cambios de ambas versiones).
4. Elimine las marcas de conflicto.
5. Realice un commit con un mensaje descriptivo en inglés, por ejemplo:

```
fix: solved conflict on README.md
```

6. Suba la rama actualizada.
7. Verifique que el Pull Request ya se puede fusionar.
8. Fusionen hacia develop.

4. Pruebas e integración final

Prueben los endpoints con Postman, Insomnia, Thunder Client o curl. Deben hacer al menos una prueba por cada endpoint: estos son los escenarios que pueden cubrir:

Prueba	Resultado esperado
GET /health	Estado de API activo
GET /notes sin registros	Lista vacía
POST /notes válido	Código 201 y nota creada
POST /notes inválido	Código 400
GET /notes/{id} existente	Código 200 y nota encontrada
GET /notes/{id} inexistente	Código 404
PUT /notes/{id} válido	Código 200 y nota actualizada
PUT /notes/{id} inexistente	Código 404, o bien Código 201 y nota creada
DELETE /notes/{id} existente	Código 200 o 204
DELETE /notes/{id} inexistente	Código 404

Luego, deben comprobar la persistencia. Para ello:

1. Creen al menos una nota.
2. Detengan los servicios con:

```
docker compose down
```

3. Inicien nuevamente los servicios:

```
docker compose up
```

4. Consulten las notas.
5. Verifiquen que la nota registrada antes del reinicio continúa disponible.

Es importante que no utilicen

```
docker compose down -v
```

antes de verificar la persistencia, porque ese comando elimina el volumen y sus datos.

Una vez hayan comprobado que todo está correcto, creen un Pull Request que vaya de la rama develop a la rama main. Luego creen un tag indicando la versión:

```
git checkout main  
git pull origin main  
git tag -a v1.0.0 -m "Laboratorio 1 finalizado"  
git push origin v1.0.0
```

Con esto, daríamos por finalizado el laboratorio.

Entrega

Deben crear un documento por pareja y dar cuenta de la ejecución de estos pasos, incluyendo capturas de pantalla, resultados de las pruebas, dificultades que hayan tenido, cosas que cada uno haya aprendido, etc: la idea es que puedan dar cuenta de su proceso. Además, el documento debe contener el link al repositorio en GitHub, pues allá se validará el control de versiones mediante los historiales de commits.

Pueden subir este archivo a la carpeta de proyecto de su equipo en OneDrive (si el grupo se subdivide en dos parejas), o enviarlo por correo a jpbustamantem@itc.edu.co.

Criterios de evaluación

Criterio	Evidencia esperada	Peso
Trabajo colaborativo con GitHub	Commits de ambos integrantes, ramas, Pull Requests y revisión cruzada	25%
Conflicto e integración	Conflicto obligatorio, solución correcta y evidencia completa	15%
API y operaciones CRUD	Endpoints funcionales, validaciones y respuestas HTTP adecuadas	25%
Uso de Docker	API y BD en contenedores, variables de entorno y volumen persistente	20%
Evidencias de desarrollo en el documento	Capturas de pantalla, explicaciones, contribuciones y aprendizajes individuales	15%